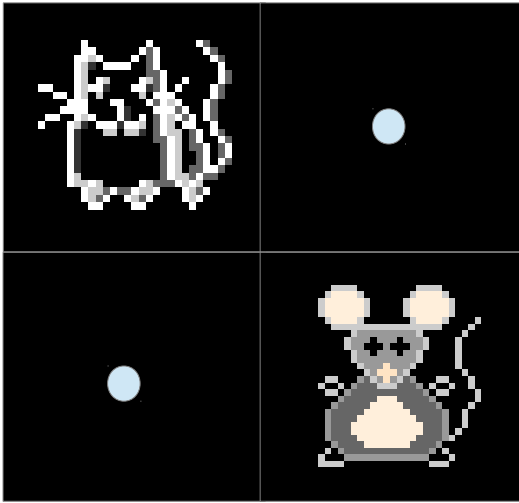


Reinforcement Learning Q-Learning



The picture at the left shows ***tiny world***, a place where a cat and a mouse chase around. There are **two** agents here. The **cat moves clockwise**. You will train the mouse to avoid getting eaten while consuming the **mouse power vitamins**. Power vitamins **regenerate** automatically on any cells not occupied by the cat or the mouse.

For the mouse two actions are possible: **Move CCW, or move CW. CCW is action 0, CW is action 1.**

The reward function is very simple: If the mouse dies reward=-100, otherwise reward=10 (since the mouse will have eaten a power vitamin)

Q1 – How many states are there in the state space? Why? How do you convert cat & mouse position to an **index** that represents the state?

Q2 – Starting with a **Q-table** containing all-zeros. Do a few rounds of Q-learning updates until you see a reasonable pattern appear. Does the resulting policy agree with what you would expect from common sense?

* use **alpha=.1** and **gamma=.5** in your update

Consider your 8-bit self-driving car

Do you think a similar strategy could be used in a real car – assuming we can appropriately digitize the road conditions a certain number of units of distance away from the car (equivalent to seeing more rows ahead in the 8-bit car simulation)

** Here we'd like to see your thoughts in terms of*

- What would the state space contain?*
- Is it manageable (the size of the state space is not too big to handle in the computer)*
- Would we expect the car to learn a reasonable policy?*
- What's the tradeoff between looking further ahead, being able to keep the state space to a reasonable size, and the performance of the car in actual driving?*
- And finally – does state-based RL seem like a good strategy for a realistic self-driving car?*

Please bring any questions to your tutorial, and submit your completed .pdf on Quercus.